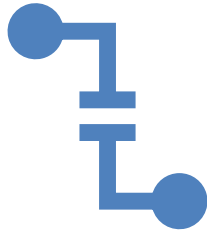


Networks and Systems Management



Device API and Automation Tools
Filipe Sousa
FEUP – GRS/M.EIC

Motivation



Why Automate Network and Systems?

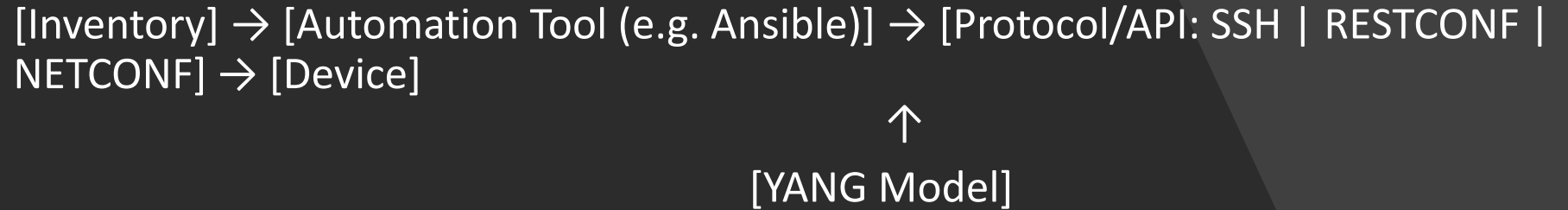
Growing complexity & number of devices
Need for repeatability, scalability, and faster changes



Examples:

Auto-configure routers
Push firmware updates
Real-time monitoring and healing

Bird's-eye View of Network Automation



- Inventory: NetBox, Nornir
- Protocols: SSH, SNMP, RESTCONF, NETCONF
- YANG defines data structure

What is YANG?



- YANG = Yet Another Next Generation
- A modeling language for defining:
 - Network configurations
 - Operational state
- Used with NETCONF/RESTCONF
- Human- and machine-readable

YANG Example

- Defines a structure for the data
 - Building blocks:
 - **module** – top of the hierarchy of nodes
 - **containers** – related nodes
 - **lists** – identifies nodes
 - **leaf** – individual attributes of a node
 - **type** – every leaf has an associated type
-

```
container interface {  
    list ethernet {  
        key "name";  
        leaf name {  
            type string;  
        }  
        leaf enabled {  
            type boolean;  
        }  
    }  
}
```

YANG for Networking

pyang -f tree ietf-ip.yang

```
module: ietf-ip
```

```
augment /if:interfaces/if:interface:
```

```
  +--rw ipv4!
```

```
    | +--rw enabled?      boolean
```

```
    | +--rw forwarding?   boolean
```

```
    | +--rw mtu?          uint16
```

```
    | +--rw address* [ip]
```

```
      | | +--rw ip                inet:ipv4-address-no-zone
```

```
      | | +--rw (subnet)
```

```
      | | | +--:(prefix-length)
```

```
      | | | | +--rw prefix-length? uint8
```

```
      | | | +--:(netmask)
```

```
      | | | | +--rw netmask?      yang:dotted-quad {ipv4-non-contiguous-netmasks}?
```

NETCONF and RESTCONF

- NETCONF: SSH transport, XML format, transactional
- RESTCONF: HTTP transport, JSON/XML, web-style access
- Both use YANG for data models
- Secure, CRUD-capable protocols

Data Model Language
(schema language)

Data Modeling
(schema)

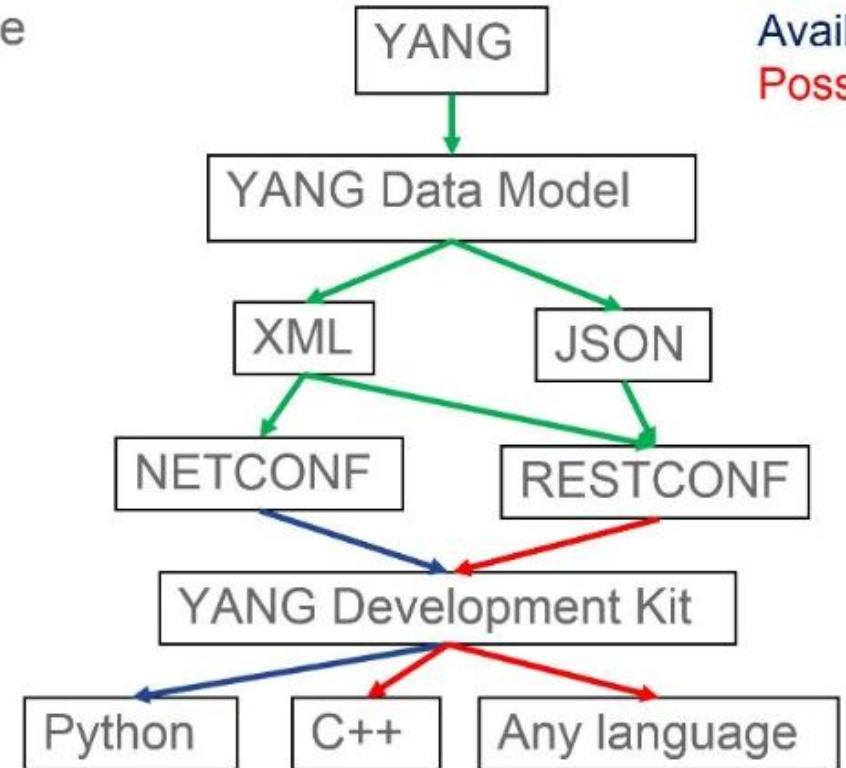
Encoding
(serialization)

Protocol

Application

Prog. Language

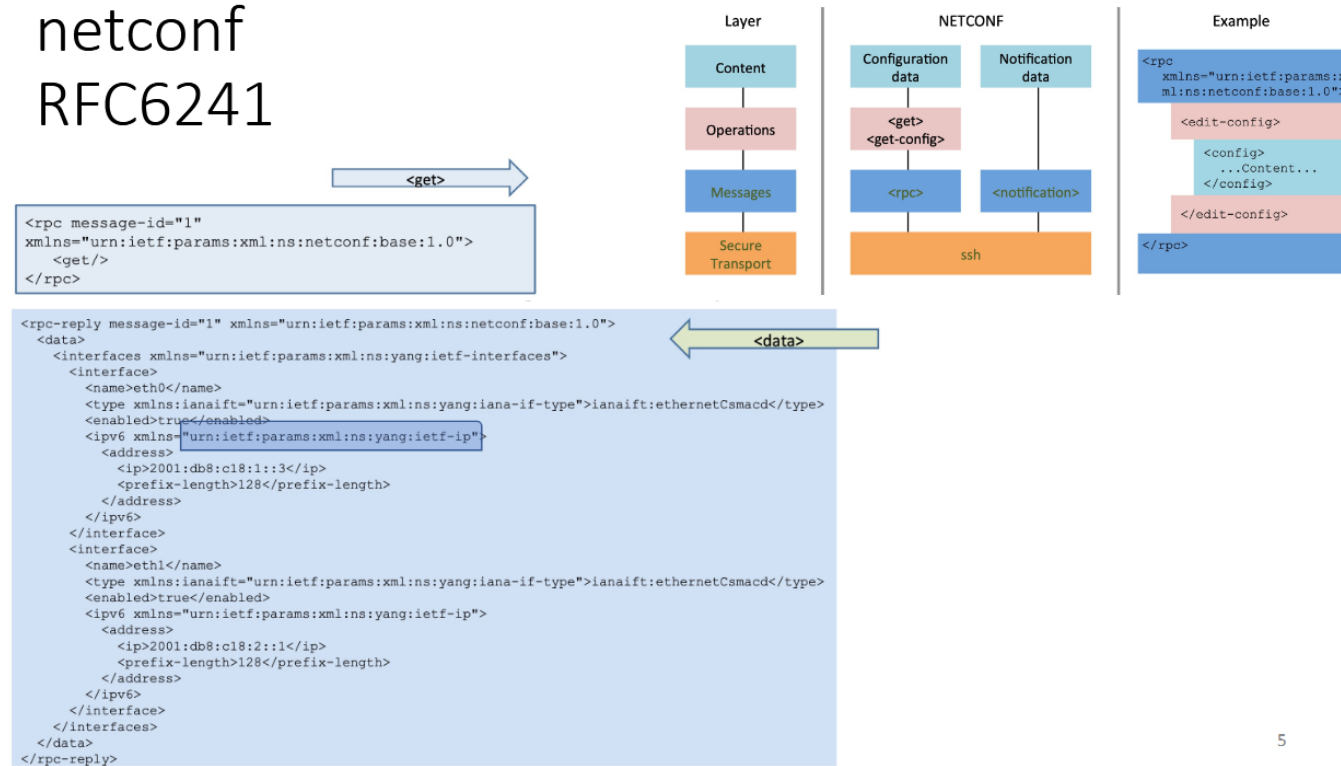
Standard
Available
Possible



NETCONF

netconf RFC6241

<https://trac.ietf.org/trac/netconf/wiki>



RESTCONF RFC8040

Acting on resources

```
Module my-interfaces {  
  {  
    namespace "com.my-interfaces";  
  
    container interfaces {  
      list interface {  
        key name;  
        leaf name { type string; }  
        leaf admin-status { type enum; }  
      }  
  
      rpc flap-interface {  
        input {  
          leaf name { type string; }  
        }  
        output {  
          leaf result { type boolean; }  
        }  
      }  
    }  
  }  
}
```

GET : Gets a resource

GET /restconf/data/my-interfaces:interfaces

GET /restconf/data/my-interfaces:interfaces/interface/<some name>

POST : Creates a resource or invoke operation

POST /restconf/operations/my-interfaces:flap-interface
+ JSON/XML Form Data (including name)

Response will have JSON/XML result

PUT : Replaces a resource

PUT /restconf/data/my-interfaces:interfaces/interface/<some name> + JSON/XML Form Data (name, admin-status)

DELETE : Removes a resource

DELETE /restconf/data/my-interfaces:interfaces/interface/<some name>

Device APIs Overview



**APIs expose device features
programmatically**



Common forms:

REST APIs (HTTP, JSON)

CLI over SSH

SNMP, NETCONF



Used for config, monitoring, control

Mikrotik REST API

- Available in RouterOS v7+
- Enable /rest service
- Uses HTTP with JSON
- Covers:
 - Firewall
 - Interfaces
 - DHCP, IP settings

Mikrotik REST API Example

<https://help.mikrotik.com/docs/spaces/ROS/pages/47579162/REST+API>

```
curl -k -u USER:PWD https://192.168.88.1/rest/system/resource
```

```
architecture-name: "arm"
bad-blocks: "0"
board-name: "CRS326-24G-2S+"
build-time: "Dec/21/2021 11:53:05"
cpu: "ARMv7"
cpu-count: "2"
cpu-load: "1"
factory-software: "6.41"
free-hdd-space: "2424832"
free-memory: "483110912"
platform: "MikroTik"
total-hdd-space: "16777216"
total-memory: "536870912"
uptime: "3m53s"
version: "7.1.1 (stable)"
write-sect-since-reboot: "798"
write-sect-total: "2553"
```

```
curl -k -u USER:PWD https://192.168.88.1/rest/ip/address
```

```
▼ 0:
  .id: "*1"
  actual-interface: "bridge"
  address: "192.168.88.1/24"
  comment: "defconf"
  disabled: "false"
  dynamic: "false"
  interface: "bridge"
  invalid: "false"
  network: "192.168.88.0"
```

Mikrotik REST API add bridge and set IP

```
curl -k -u USER:PWD -X PUT
https://192.168.88.1/rest/interface/bridge --data '{"name":
"test123"}' -H "content-type: application/json"
```

<https://192.168.88.1/rest/interface/bridge/test123>

```
.id:                "*1E"
actual-mtu:         "1500"
ageing-time:        "5m"
arp:                "enabled"
arp-timeout:        "auto"
auto-mac:           "true"
dhcp-snooping:      "false"
disabled:           "false"
fast-forward:       "true"
forward-delay:      "15s"
igmp-snooping:      "false"
l2mtu:              "65535"
mac-address:        "2A:AC:ED:F4:D6:B4"
max-message-age:    "20s"
mtu:                "auto"
name:               "test123"
priority:           "0x8000"
protocol-mode:      "rstp"
running:            "true"
status:             "up"
```

```
#curl -k -u USER:PWD -X PUT
https://192.168.88.1/rest/ip/address --data '{"address":
"192.168.111.111", "interface": "test123"}' -H "content-
type: application/json"
```

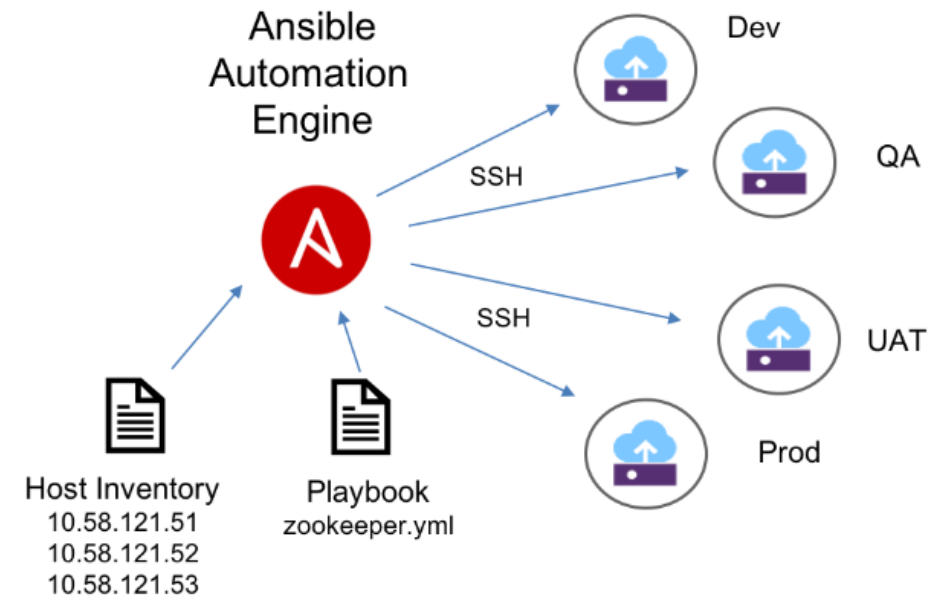
<https://192.168.88.1/rest/ip/address>

```
▼ 0:
  .id:                "*1"
  actual-interface:    "bridge"
  address:             "192.168.88.1/24"
  comment:             "defconf"
  disabled:            "false"
  dynamic:             "false"
  interface:           "bridge"
  invalid:             "false"
  network:             "192.168.88.0"

▼ 1:
  .id:                "*2"
  actual-interface:    "test123"
  address:             "192.168.111.111/32"
  disabled:            "false"
  dynamic:             "false"
  interface:           "test123"
  invalid:             "false"
  network:             "192.168.111.111"
```

Automation Tools Overview

- Ansible: Agentless, YAML playbooks
- SaltStack: Real-time automation
- Terraform: Infra as Code (cloud, virtual)
- Nornir: Python-native, network-focused



YANG + Ansible + RESTCONF

- Workflow:
 1. Get supported YANG models from device
 2. Use Ansible's restconf plugin or uri module
 3. Structure JSON/YAML to match model

Ansible Config via RESTCONF

```
- name: Configure interface
hosts: routers
tasks:
  - name: Set IP address
    uri:
      url: "https://{{ inventory_hostname
}}/restconf/data/interfaces/interface=ether1"
      method: PATCH
      headers:
        Content-Type: application/yang-data+json
      body: >
        {
          "interface": {
            "name": "ether1",
            "enabled": true,
            "ip": "192.168.88.1/24"
          }
        }
      }
```

Ansible quick start

- On the config host
 - apt install ansible
 - ansible.cfg
 - [defaults]
 - inventory =
/home/gors/ansible/hosts
 - Hosts
 - [targets]
 - m-gors-B
 - m-gors-C

```
gors@gors-A:~/ansible$ ansible all --list-hosts
hosts (2):
  m-gors-B
  m-gors-C

gors@gors-A:~/ansible$ ansible all -m ping
m-gors-B | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
m-gors-C | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
```

Ansible upload configuration files

```
- hosts: targets
  tasks:
    - name: Copy file with owner and permissions
      ansible.builtin.copy:
        src: my-config-file
        dest: ~/foo.conf
        owner: gors
        group: gors
        mode: '0644'
```

```
gors@gors-A:~/ansible$ ssh m-gors-B 'ls ~/foo.conf'
/home/gors/foo.conf
```

```
ansible > ≡ my-config-file.j2
1  This is {{ inventory_hostname }}
2
```

```
ansible > ! deploytemplate.yml
1  - hosts: targets
2  tasks:
3    - name: deploy template
4      template:
5        src: my-config-file.j2
6        dest: ~/foo2.conf
7
```

```
gors@gors-A:~/ansible$ ssh m-gors-B 'cat ~/foo2.conf'
This is m-gors-B
```

Challenges in Practice

- Vendor-specific quirks and APIs
- Keeping YANG models in sync
- Credential management
- Versioning and rollback



Key Takeaways

Device APIs and YANG enable scalable automation

Mikrotik, Cisco, Juniper provide APIs

Tools like Ansible make automation accessible

NETCONF/RESTCONF standardize config access

Q&A / Discussion

- Have you used these tools or APIs?
- What devices are relevant in your setup?



Further Reading & References

- RFC 7950 – The YANG 1.1 Data Modeling Language
- RFC 6241 – NETCONF Protocol
- RFC 8040 – RESTCONF Protocol
- Mikrotik REST API Docs: <https://help.mikrotik.com/docs/display/ROS/REST+API>
- Cisco DevNet: <https://developer.cisco.com/>
- Ansible Network Guide: <https://docs.ansible.com/network/>
- Juniper Automation: https://www.juniper.net/documentation/en_US/junos-automation/
- YANG Models Repository: <https://yangcatalog.org>

